

CROP MAX AI — MIDTERM REFLECTION

Progress Report, Challenges, and Forward Path

Intern ID: TBI-26101097

Period: Weeks 1–4 Overview

Reflection Report

🚀 3 CORE THINGS BUILT OR LEARNED (WEEKS 1–4)

1. Modular Responsive React Components in Next.js (Tailwind CSS)

What was built: Designed a responsive, modern landing page with a unified design system. Created a reusable `Card.jsx` component that displays crop details, optimization status badges (info, success, warning, danger), and responsive styles that expand fluidly based on screen widths.

```
export default function Card({ title, description, badgeText, badgeType = "info", icon, actionText, onClick }) {
  const badgeColorMap = {
    info: "bg-blue-50 text-blue-700 border-blue-200/50 dark:bg-blue-950/40 dark:text-blue-300 dark:border-blue-800/30",
    success: "bg-emerald-50 text-emerald-700 border-emerald-200/50 dark:bg-emerald-950/40 dark:text-emerald-300 dark:borde",
    warning: "bg-amber-50 text-amber-700 border-amber-200/50 dark:bg-amber-950/40 dark:text-amber-300 dark:border-amber-80",
    danger: "bg-red-50 text-red-700 border-red-200/50 dark:bg-red-950/40 dark:text-red-300 dark:border-red-800/30",
  };
  return (
    <div className="group bg-white dark:bg-zinc-900 rounded-2xl border border-zinc-200/60 hover:shadow-lg transition-all f
      ...
    </div>
  );
}
```

2. REST API Architecture with Node.js & Express.js

What was built: Built a robust Express backend supporting a modular crop router. Designed 7 REST endpoints for CRUD operations (GET/POST/PUT/DELETE), structured metric computations (Projected Income aggregations), and wildcard search logic on MySQL crops rows.

```
// GET /api/crops/stats - Calculate aggregate metrics based on crop entries
router.get('/stats', async (req, res, next) => {
  try {
    const list = await prisma.crop.findMany();
    const totalCrops = list.length;
    let projectedIncome = list.reduce((sum, crop) => {
      let multiplier = crop.unit.toLowerCase() === 'tons' ? 20000 : 2000;
      return sum + (crop.quantity * multiplier * (crop.status.includes("Processing") ? 1.55 : 1.18));
    }, 0);
    res.status(200).json({ totalCrops, projectedIncome: formatCurrency(projectedIncome) });
  } catch (error) { next(error); }
});
```

CROP MAX AI — MIDTERM REFLECTION

Progress Report, Challenges, and Forward Path

Intern ID: TBI-26101097

Period: Weeks 1–4 Overview

Reflection Report

3. Interactive Chrome DevTools Request Console in UI

What was built: Integrated a real-time HTTP monitoring console directly at the bottom of the dashboard. This intercepts outgoing frontend fetch calls, logs timestamps, methods (GET/POST/PUT/DELETE), endpoints, latency (ms), and response status codes, simulating a DevTools Network tab for live visual confirmation.

```
const loggedFetch = useCallback(async (url, options = {}) => {
  const startTime = Date.now();
  const method = options.method || "GET";
  const logId = Math.random().toString();
  setNetworkLogs(prev => [{ id: logId, url: url.replace("http://localhost:5000", ""), method, status: "PENDING" }, ...prev]);
  try {
    const response = await fetch(url, options);
    setNetworkLogs(prev => prev.map(log => log.id === logId ? { ...log, status: response.status, latency: Date.now() - startTime } : log));
    return response;
  } catch (error) { ... }
}, []);
```

CHALLENGE FACED & HOW IT WAS DEBUGGED

The Challenge: During the automation of the CRUD verification screenshots in Week 5 using a headless Playwright browser subagent, the deletion test step failed. Clicking the delete button triggered a native browser confirmation modal (`window.confirm()`). In a headless environment, these prompts have no user input and return `false` by default, halting the REST call and screenshot capture sequence.

The Debugging Process:

- Locating the roadblock:** Analyzed the backend task logs and saw that the DELETE endpoint was never being hit from the automated agent, indicating the action was stopped on the client.
- Testing standard overrides:** Attempted to configure Playwright listeners (`page.on('dialog')`) inside the subagent context, but the headless configuration of the agent tool blocked native listener injections.
- Applying a developer bypass:** Temporarily commented out the client-side confirmation check in `handleDeleteCrop`:

```
// Temporary bypass applied during automated screenshots:
// if (!confirm("Are you sure you want to delete this harvest record?")) { return; }
```

This allowed the subagent to click the button and immediately trigger the DELETE REST API. The screenshots were captured perfectly, and the confirm check was immediately restored to ensure user-facing safety in production.

CROPMAX AI — MIDTERM REFLECTION

Progress Report, Challenges, and Forward Path

GOALS AND MILESTONES (WEEKS 5–9)

As we pivot from core database integrations into advanced product features, the roadmap for the remaining weeks is structured as follows:

- Week 5 — Database Integration & Seeding (Current)**
Migrate in-memory state to local MySQL database using Prisma. Seed initial mock datasets (User, Categories, Crops) and verify database CRUD transactions using automated browser captures.
- Week 6 — Authentication & User Access Control (JWT)**
Implement secure User Registration and Login. Secure backend API endpoints with JWT verification middleware and store user profiles in MySQL. Configure frontend state context to load only user-specific crop harvests.
- Week 7 — Price Analytics & Visual Dashboards (Chart.js / Recharts)**
Create interactive line and bar charts showing historical APMC mandi price trends, regional yield distributions, and dynamic comparisons showing the profit spread between raw selling and processed value addition.
- Week 8 — SMS Alert Engine & Regional mandi Matcher**
Integrate Twilio or similar text notification system. Allow farmers to sign up for price alerts. If mandi rates for a registered crop fluctuate by more than 15%, broadcast automated price warning SMS alerts.
- Week 9 — Cloud Deployment & Performance Caching**
Deploy Next.js frontend to Vercel and Express.js backend to Render/Railway. Migrate MySQL database to a cloud instance (e.g. Supabase, PlanetScale/TiDB). Add API caching layer to minimize database load times and optimize visual responsiveness.

Report prepared by Intern TBI-26101097. Checked and verified locally in workspace directory c:\Users\Lenovo\Desktop\CropMax AI.